

GCS Mission Planner — Implementation Report

Executive Summary

This Flutter GCS (Mission Planner) implementation is a **map-first tactical UI shell** with sidebar navigation, top status bar, right telemetry panel, and a bottom mission dock. **Core UI flows and simulation-based telemetry update correctly**, but **real vehicle integration (MAVLink), calibration, mission upload, and hardware control are not implemented**—most operational actions are demo/placeholder.

1) Current Scope & Runtime Environment

- **Platform:** Flutter Web (Chrome) in this workspace
- **UI Style:** Dark tactical GCS theme (panel borders, glow accents, frosted top bar)
- **Data Source:** **Simulated telemetry** + placeholder mission actions
- **Real FC Link:** **Not present** (no MAVLink parsing/transport)

2) Application Layout & Main Components

2.1 Top Status Bar (Header Strip)

Purpose

- Shows connection/health indicators and key status:
- Link indicator
- GPS label
- Battery label
- Flight mode

Controls

- Tools button (opens Tools sheet)
- Telemetry toggle (opens/closes right telemetry panel)
- Close page (returns to map when on overlay pages)

Works properly

- Status text updates live (from the shared status model driven by the simulator)
- Tools sheet opens reliably
- Telemetry panel toggle works (open/close)

Not real yet

- Link/connection status is not backed by any real transport or vehicle connection

2.2 Left Sidebar (Navigation Hub)

Sections

- Map
- Mission
- Logs
- Settings
- Tools

Works properly

- Selected item highlight
- Collapse/expand via chevron handle
- Tools opens tool sheet
- Navigation switches content sections

Not real yet

- Some pages are UI scaffolds (depends on route/page implementation)

2.3 Center Map (Primary Area)

Purpose

- Primary operational surface: basemap, overlays, waypoints, mission geometry, vehicle marker

Works properly

- Map renders (`flutter\_map`) and responds to interactions
- Basemap switching (dark/satellite) works
- Airspace overlay toggle works (demo NFZ logic)
- Mission geometry toggle works (route + waypoint markers)
- Vehicle marker + heading updates smoothly (simulated)
- Follow/tracking updates camera position (throttled for web performance)
- Mode-based overlays (banner/tag) display when mode changes
- Zoom rail works and is positioned to track the right telemetry strip (when used from the GCS shell)

Not real yet

- No MAVLink vehicle position/attitude
- No real airspace feeds; NFZ is demo logic

2.4 Right Telemetry Panel (Collapsible Overlay)

Purpose

- Vertical telemetry deck (cards) with core metrics:
- Altitude (MSL)
- Speed
- Climb rate
- Battery %
- GNSS sats
- Heading
- HDOP

Works properly

- Collapsible handle works (panel open/closed)
- Width clamps to avoid overflow
- Telemetry values update in real time (simulated)

Not real yet

- Not backed by MAVLink messages

2.5 Bottom Dock (Mission Controls + Logs + Graph Stub)

States

- Compact: toolbar row
- Expanded: toolbar + logs + graph stub

Toolbar buttons present

- Start Mission
- Load
- Save
- Pause (placeholder)
- RTH (placeholder)
- Waypoints (opens waypoint property sheet)
- Summary (opens mission summary sheet)
- Track (toggles follow/tracking)
- Emergency Stop (UI-only)

Works properly

- Expand/collapse dock works
- Horizontal toolbar scroll prevents overflow
- Buttons are sized tighter to keep fit
- Logs list renders (demo lines)
- Graph area renders stub “ALT live graph”

Not real yet

- Start/Pause/RTH/E-Stop do not send commands to a vehicle
- Load/Save are not a full mission file pipeline to/from FC (currently demo dialogs)

3) Mission / Waypoint Features

3.1 Waypoint Creation

Current UX

- Long-press map to add waypoint (demo behavior)
- “Add waypoint” appears in map actions menu

Works properly

- Waypoint list/route visuals are shown when mission geometry is enabled
- Demo NFZ violation blocks mission start (UI gating)

Not real yet

- No upload/download via MAVLink mission protocol
- No persistence to vehicle/autopilot

3.2 Mission Summary & Start Gating

Summary sheet shows

- Waypoint count
- Pre-arm state (demo)
- Airspace violation state (demo)
- Ready/Blocked

Works properly

- Ready/Blocked updates from demo conditions (NFZ + pre-arm)
- Waypoint editor opens (property sheet)

Not real yet

- No true pre-arm checks from autopilot

- No mission start/monitor via real vehicle

3.3 Map Actions Menu (“More actions”)

Includes

- Add waypoint
- Map layers
- Arm/Disarm (demo)
- Flight mode select (dialog)
- New mission wizard
- Survey grid (placeholder)
- Import mission (placeholder)

Works properly

- Menu and dialogs behave correctly
- Mode selection updates UI mode state
- Arm/disarm toggles simulated armed state
- Map layers sheet toggles overlays

Not real yet

- No arm/disarm/mode commands sent to autopilot

4) Tools Sheet (Secondary Panels / Utilities)

Access

- From top bar Tools icon
- From sidebar Tools item

Entries

- Telemetry
- Camera
- Manual control
- Airspace
- Replay
- Plugins
- Vehicles
- Vehicle setup

Works properly

- Sheet opens and navigates to routes

Not real yet

- Most tools/routes are UI scaffolding unless wired to real data/services

5) Responsiveness / Non-overflow Handling (Current Quality)

Solid behaviors

- Sidebar collapse/expand transitions are smooth
- Telemetry panel width clamps to prevent overflow
- Dock toolbar scroll prevents RenderFlex overflow

- Map update cadence throttled for web performance

Known limitation

- This is still an overlay-heavy shell (map + panels). It is stable visually, but not yet a fully “native” non-overlay grid layout.

6) What’s Missing for “Real GCS” Operation (Critical Gaps)

6.1 Vehicle Link (Required)

- MAVLink transport: USB serial / telemetry radio / UDP
- MAVLink parsing and message routing
- Command/ack handling
- Heartbeat and link health computation

6.2 Calibration (Required for real setup)

- Sensor calibration flows via MAVLink commands
- Progress / instruction feedback (STATUSTEXT, ACKs)
- Parameter access (read/write, persistence, reboot workflows)

6.3 Mission Protocol (Required)

- Upload/download mission items
- Start/stop/monitor mission
- Geofence/failsafe handling

6.4 Safety & Reliability (Strongly recommended)

- State gating (disallow calibration while armed)
- Failsafe surfaces (link loss, EKF errors, GPS loss)
- Proper emergency stop behavior (kill/disarm) where supported

7) Feature Status Matrix (■ / ■ / ■)

Area	Feature	Status	Shell	Top status bar + actions	Map	Basemap + overlays	Vehicle marker + tracking	Zoom rail	Mission	Waypoint visuals + route	Mission start gating (NFZ/prearm)	Mission upload/download	Telemetry	Right panel + live updates	Dock	Toolbar + logs + graph stub	Commands	Arm/disarm/mode	Calibration	Sensors/radio/params	Connectivity	MAVLink transport

Legend:

- ■ Working properly (in current implementation)
- ■ Present but demo/placeholder (not real vehicle-backed)
- ■ Not implemented

8) Conclusion

The GCS you created is a **functional, polished UI shell** with **working layout behaviors, navigation, map rendering, and simulation-driven telemetry**. It is not yet a real operational GCS because it lacks the **MAVLink connectivity layer, calibration workflows, and mission protocol integration** needed to communicate with PX4/ArduPilot flight controllers.